

Exercise 7 - Comparing penalized regression estimators

Florescu Andrei - 12242987

2024

Task 1

```
set.seed(12242987)
library(knitr)
```

```
## Warning: package 'knitr' was built under R version 4.4.2
```

1. Write your own function for the lasso using the shooting algorithm

```
soft_threshold <- function(z, gamma) {
  return(sign(z) * max(0, abs(z) - gamma))
}

lasso_shooting <- function(X, y, lambda, tol = 0.000001, max_iter = 1000) {
  n <- nrow(X)
  p <- ncol(X)
  X_mean <- colMeans(X)
  y_mean <- mean(y)
  X_centered <- scale(X, center = TRUE, scale = FALSE)
  y_centered <- y - y_mean

  # initialize coefs
  beta <- rep(0, p)
  beta_old <- beta

  ##### Shooting algorithm #####

  # compute a_j (2 * sum(x_ij^2))
  a <- 2 * colSums(X_centered^2)

  for (iter in 1:max_iter) {
    for (j in 1:p) {
      # compute c_j
      r <- y_centered - X_centered %*% beta + beta[j] * X_centered[, j]
      c_j <- 2 * sum(X_centered[, j] * r)

      # soft update
      beta[j] <- soft_threshold(c_j / a[j], lambda / a[j])
    }

    # check for stopping
```

```

    if (max(abs(beta - beta_old)) < tol) {
      break
    }
    beta_old <- beta
  }

  # compute intercept
  intercept <- y_mean - sum(X_mean * beta)

  # return coefficients, intercept and nr of iterations
  return(list(intercept = intercept, coefficients = beta, iterations = iter))
}

## test to see if it works
n <- 100
p <- 5
X <- matrix(rnorm(n * p), n, p)
true_beta <- c(1, -5, -3, 2, 1) # random values selected as coeffs
y <- X %%% true_beta + rnorm(n)
lambda <- 0.5
result <- lasso_shooting(X, y, lambda)
result

```

```

## $intercept
## [1] -0.009275647
##
## $coefficients
## [1] 0.9948361 -5.0440608 -3.0821463 1.9602295 0.9327584
##
## $iterations
## [1] 7

```

2. Write a function which computes the lasso using your algorithm for a vector of lambdas

```

lasso_for_lambda_vector <- function(X, y, lambda_vector, tol = 0.000001, max_iter = 1000) {
  beta_matrix <- matrix(0, nrow = ncol(X)+1, ncol = length(lambda_vector))
  for (i in 1:length(lambda_vector)) { # nolint: seq_linter.
    lambda <- lambda_vector[i]
    result <- lasso_shooting(X, y, lambda, tol, max_iter)
    beta_matrix[, i] <- c(result$intercept, result$coefficients)
  }
  return(list(coefficients = beta_matrix, lambda = lambda_vector))
}

# test for a sample lambda vector
lambda_vector <- c(0.1, 0.3, 0.5, 0.7, 0.9)
result <- lasso_for_lambda_vector(X, y, lambda_vector)
result

```

```

## $coefficients
##           [,1]           [,2]           [,3]           [,4]           [,5]
## [1,] -0.009598036 -0.009436841 -0.009275647 -0.009114452 -0.008953258
## [2,] 0.996970187 0.995903119 0.994836051 0.993768983 0.992701915

```

```
## [3,] -5.046120249 -5.045090520 -5.044060790 -5.043031060 -5.042001330
## [4,] -3.084934105 -3.083540184 -3.082146263 -3.080752342 -3.079358422
## [5,]  1.962728572  1.961479048  1.960229525  1.958980001  1.957730477
## [6,]  0.935525769  0.934142088  0.932758407  0.931374726  0.929991045
##
## $lambda
## [1] 0.1 0.3 0.5 0.7 0.9
```

3. Compare the performance and output of your functions against the lasso implementation from glmnet.

```
library(glmnet)
```

```
## Warning: package 'glmnet' was built under R version 4.4.2
```

```
## Loading required package: Matrix
```

```
## Loaded glmnet 4.1-8
```

```
library(microbenchmark)
```

```
shooting_result <- lasso_for_lambda_vector(X, y, lambda_vector)
```

```
# Fit LASSO Using glmnet
```

```
glmnet_result <- glmnet(X, y, alpha = 1, lambda = lambda_vector, standardize = TRUE, intercept = TRUE)
```

```
# Compare Coefficients
```

```
cat("Comparison of Coefficients (Shooting vs glmnet):\n")
```

```
## Comparison of Coefficients (Shooting vs glmnet):
```

```
shooting_coef <- shooting_result$coefficients[-1, ] # Exclude intercept
```

```
glmnet_coef <- as.matrix(coef(glmnet_result))[-1, ] # Exclude intercept
```

Shooting lasso coefficients

```
shooting_coef
```

```
##           [,1]      [,2]      [,3]      [,4]      [,5]
## [1,] 0.9969702 0.9959031 0.9948361 0.9937690 0.9927019
## [2,] -5.0461202 -5.0450905 -5.0440608 -5.0430311 -5.0420013
## [3,] -3.0849341 -3.0835402 -3.0821463 -3.0807523 -3.0793584
## [4,]  1.9627286  1.9614790  1.9602295  1.9589800  1.9577305
## [5,]  0.9355258  0.9341421  0.9327584  0.9313747  0.9299910
```

glmnet coefficients

```
glmnet_coef
```

```
##           s0           s1           s2           s3           s4
## V1 0.0000000 0.2192448 0.4415914 0.6639558 0.8863209
## V2 -4.0945785 -4.2973290 -4.5111744 -4.7253591 -4.9395431
## V3 -1.8990525 -2.1299426 -2.4021206 -2.6755249 -2.9489290
## V4  0.8718135  1.1012199  1.3472030  1.5936630  1.8401232
## V5  0.0000000  0.0000000  0.2619466  0.5316550  0.8013634
```

Execution time comparison

```
benchmark_result <- microbenchmark(  
  Shooting = lasso_for_lambda_vector(X, y, lambda_vector),  
  glmnet = glmnet(X, y, alpha = 1, lambda = lambda_vector, standardize = TRUE, intercept = TRUE),  
  times = 10  
)  
benchmark_result
```

Unit: microseconds

##	expr	min	lq	mean	median	uq	max	neval
##	Shooting	1551.9	1565.0	1870.76	1833.45	2114.4	2479.0	10
##	glmnet	981.5	1135.2	1739.92	1238.55	1339.0	6178.5	10

4. Write a function to perform 10-fold cross-validation for the lasso using MSE as the performance measure.

lasso shooting results

```
cv_lasso_shooting <- function(X, y, lambda_vec, folds = 10, tol = 1e-6, max_iter = 1000) {  
  n <- nrow(X)  
  fold_ids <- sample(rep(1:folds, length.out = n)) # random fold ids  
  n_lambda <- length(lambda_vec)  
  mse_matrix <- matrix(0, nrow = folds, ncol = n_lambda)  
  
  ### CV ###  
  for (k in 1:folds) {  
    # split data  
    train_index <- which(fold_ids != k)  
    test_index <- which(fold_ids == k)  
  
    X_train <- X[train_index, , drop = FALSE]  
    y_train <- y[train_index]  
    X_test <- X[test_index, , drop = FALSE]  
    y_test <- y[test_index]  
  
    # calculate lasso  
    shooting_result <- lasso_for_lambda_vector(X_train, y_train, lambda_vec, tol, max_iter)  
  
    # get coefs and intercept  
    betas <- shooting_result$coefficients[-1, ] # Coefficients  
    intercepts <- shooting_result$coefficients[1, ] # Intercepts  
  
    # predict on test set and calculate mse  
    for (l in 1:n_lambda) {  
      y_pred <- intercepts[l] + X_test %*% betas[, l]  
      mse_matrix[k, l] <- mean((y_test - y_pred)^2)  
    }  
  }  
  
  # average mse at the end  
  avg_mse <- colMeans(mse_matrix)  
  
  # get lambda with lowest mse and rmse
```

```

lambda_min_mse <- lambda_vec[which.min(avg_mse)]
rmse <- sqrt(avg_mse)
lambda_min_rmse <- lambda_vec[which.min(rmse)]

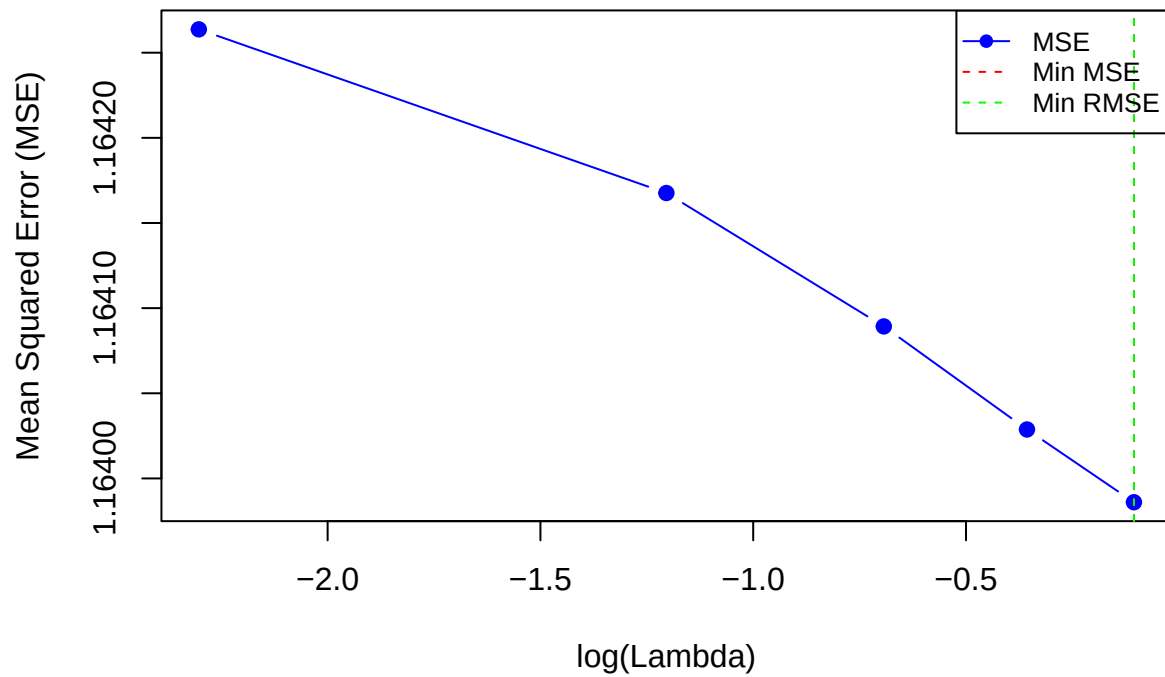
# plot mse vs Lambda
plot(log(lambda_vec), avg_mse,
     type = "b", pch = 19, col = "blue",
     xlab = "log(Lambda)", ylab = "Mean Squared Error (MSE)", main = "10-Fold Cross-Validation"
)
abline(v = log(lambda_min_mse), col = "red", lty = 2)
abline(v = log(lambda_min_rmse), col = "green", lty = 2)
legend("topright",
     legend = c("MSE", "Min MSE", "Min RMSE"),
     col = c("blue", "red", "green"), lty = c(1, 2, 2), pch = c(19, NA, NA), cex = 0.8
)

# return results
list(
  lambda_min_mse = lambda_min_mse,
  lambda_min_rmse = lambda_min_rmse,
  mse = avg_mse,
  rmse = rmse,
  lambda = lambda_vec
)
}

cv_result <- cv_lasso_shooting(X, y, lambda_vector)

```

10-Fold Cross-Validation

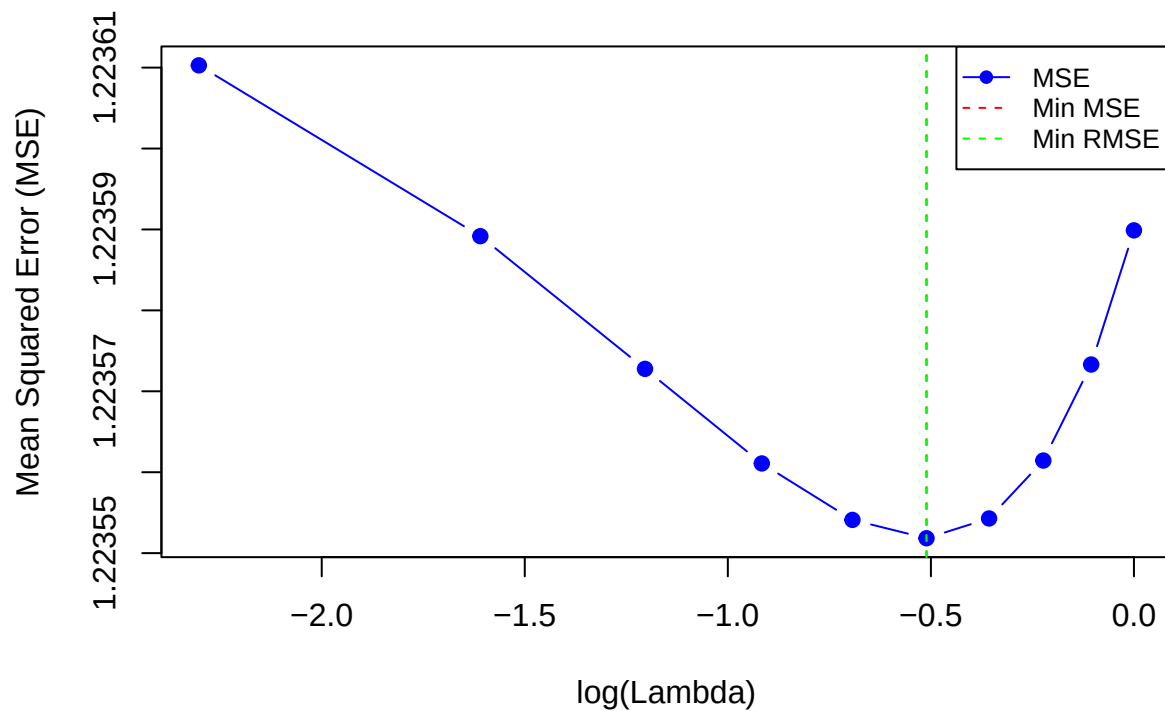


```
cv_result
```

```
## $lambda_min_mse
## [1] 0.9
##
## $lambda_min_rmse
## [1] 0.9
##
## $mse
## [1] 1.164264 1.164168 1.164089 1.164029 1.163986
##
## $rmse
## [1] 1.079011 1.078966 1.078930 1.078902 1.078882
##
## $lambda
## [1] 0.1 0.3 0.5 0.7 0.9
```

```
lambda_vec <- seq(0.1, 1, by = 0.1)
cv_result <- cv_lasso_shooting(X, y, lambda_vec)
```

10-Fold Cross-Validation

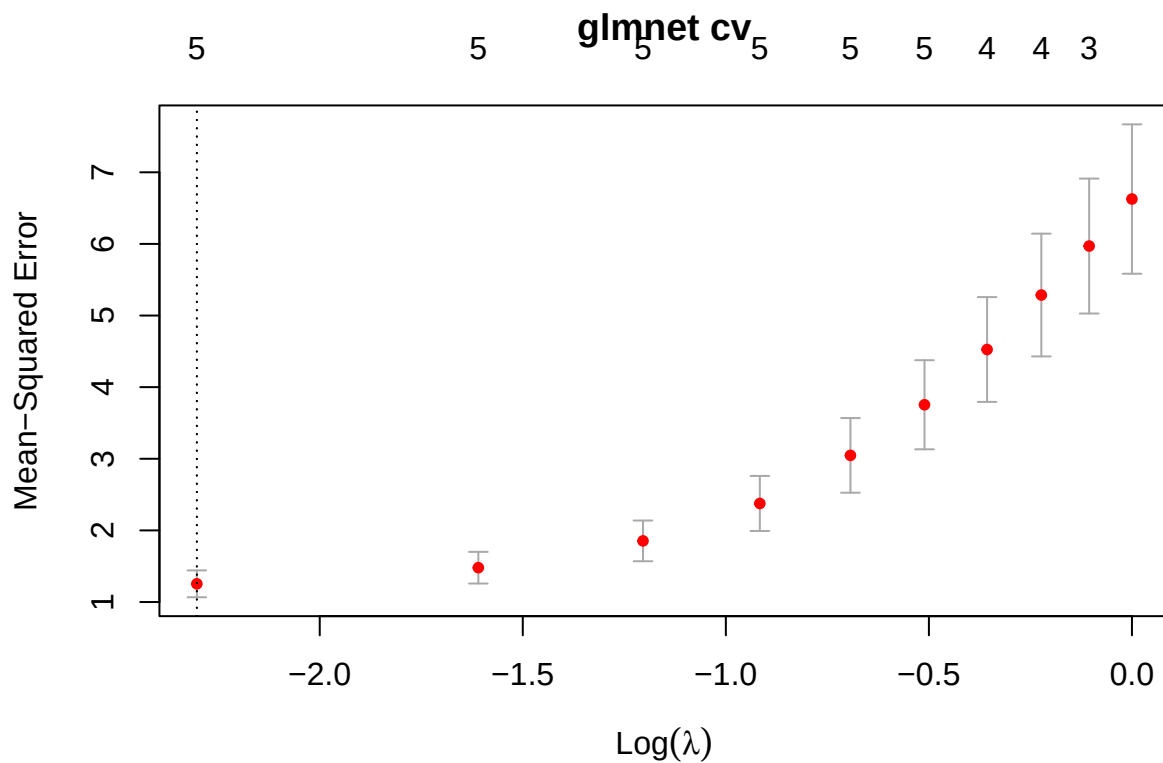


cv_result

```
## $lambda_min_mse
## [1] 0.6
##
## $lambda_min_rmse
## [1] 0.6
##
## $mse
## [1] 1.223610 1.223589 1.223573 1.223561 1.223554 1.223552 1.223554 1.223561
## [9] 1.223573 1.223590
##
## $rmse
## [1] 1.106169 1.106160 1.106152 1.106147 1.106144 1.106143 1.106144 1.106147
## [9] 1.106152 1.106160
##
## $lambda
## [1] 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0
```

glmnet results

```
cv_glmnet <- cv.glmnet(X, y, alpha = 1, lambda = lambda_vec, nfolds = 10)
plot(cv_glmnet, main = "glmnet cv")
```



Task 2

```
library(ISLR)

## Warning: package 'ISLR' was built under R version 4.4.2

# open dataset
data("Hitters")
df <- na.omit(Hitters)
y <- df$Salary
X <- model.matrix(Salary ~ ., data = df)[, -1]

# split data 70/30
train_index <- sample(1:n, size = 0.7 * n)
X_train <- X[train_index, ]
X_test <- X[-train_index, ]
y_train <- y[train_index]
y_test <- y[-train_index]

kable(head(df), caption = "Hitters dataset (head)")
```

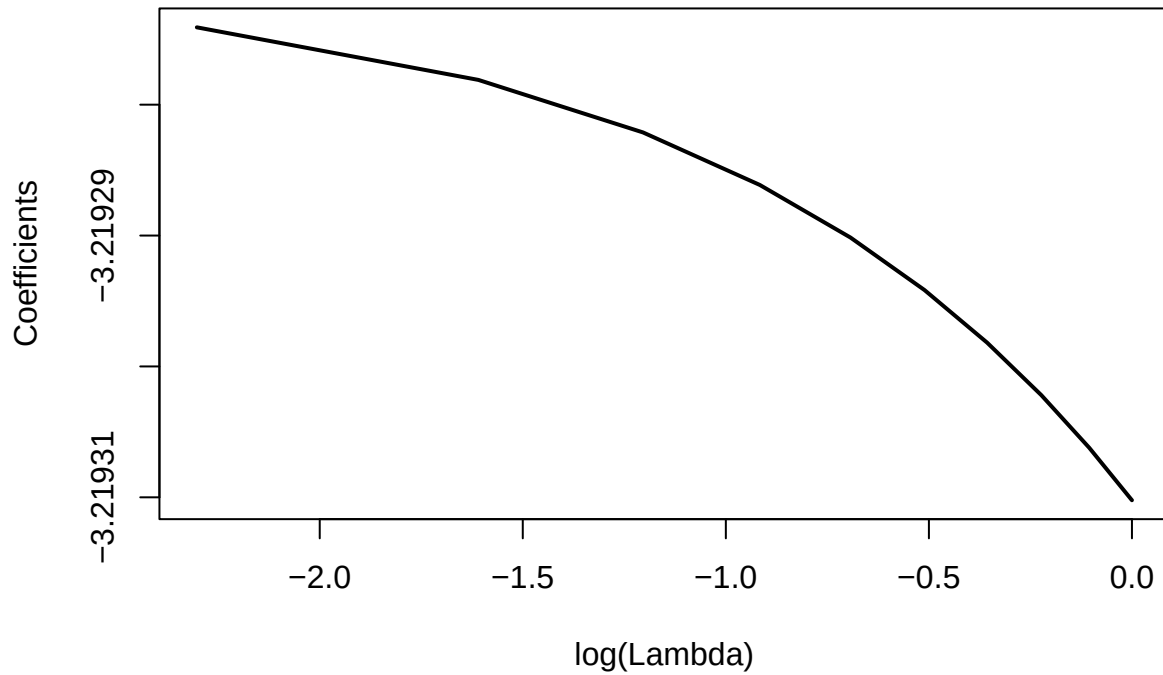
Table 1: Hitters dataset (head)

	AtBat	Hits	HmR	Runs	RBI	Walks	Years	CAtBat	CHits	CHmR	CRuns	C RBI	CWalks	League	Division	Outs	Assists	Errors	Salary	NewLeague
-Alan Ashby	315	81	7	24	38	39	14	344	835	69	321	414	375	N	W	632	43	10	475.0	N
-Alvin Davis	479	130	18	66	72	76	3	162	457	63	224	266	263	A	W	880	82	14	480.0	A
-Andre Dawson	496	141	20	65	78	37	11	562	1575	225	828	838	354	N	E	200	11	3	500.0	N
-Andres Galaraga	321	87	10	39	42	30	2	396	101	12	48	46	33	N	E	805	40	4	91.5	N
-Alfredo Griffin	594	169	4	74	51	35	11	440	1133	19	501	336	194	A	W	282	421	25	750.0	A
-Al Newman	185	37	1	23	8	21	2	214	42	1	30	9	24	N	E	76	127	7	70.0	A

1. Use your lasso function to decide which lambda is best here. Plot also the whole path for the coefficients.

```
lambda_vector <- seq(0.1, 1, by = 0.1)
result <- lasso_for_lambda_vector(X_train, y_train, lambda_vector)
beta_matrix <- result$coefficients[-1, ] # exclude intercept
lambda_values <- result$lambda
plot(log(lambda_values), beta_matrix[1, ], type = "l", col = 1, lwd = 2,
     xlab = "log(Lambda)", ylab = "Coefficients", main = "LASSO Coefficient Paths")
for (j in 2:nrow(beta_matrix)) {
  lines(log(lambda_values), beta_matrix[j, ], col = j, lwd = 2)
}
```

LASSO Coefficient Paths



result

```
## $coefficients
##           [,1]           [,2]           [,3]           [,4]           [,5]
## [1,]  82.96235341  82.96866567  82.97497793  82.98129019  82.98760245
## [2,] -3.21927409 -3.21927811 -3.21928212 -3.21928614 -3.21929015
## [3,]  12.99853329  12.99845537  12.99837745  12.99829953  12.99822162
## [4,] -13.73756318 -13.73709780 -13.73663243 -13.73616705 -13.73570168
## [5,] -0.06378459 -0.06376022 -0.06373586 -0.06371150 -0.06368714
## [6,]   2.81427335   2.81420236   2.81413137   2.81406038   2.81398939
## [7,]   5.09226873   5.09221955   5.09217036   5.09212118   5.09207199
## [8,]  24.02650863  24.02595770  24.02540677  24.02485585  24.02430492
## [9,] -0.66783349 -0.66783390 -0.66783431 -0.66783472 -0.66783513
## [10,]  1.67431372  1.67431064  1.67430757  1.67430449  1.67430141
## [11,]  1.70450181  1.70448420  1.70446658  1.70444896  1.70443134
## [12,]  0.50875778  0.50879080  0.50882382  0.50885685  0.50888987
## [13,]  1.09839293  1.09838515  1.09837738  1.09836960  1.09836183
## [14,] -0.33620971 -0.33621484 -0.33621998 -0.33622511 -0.33623025
## [15,] 167.45211141 167.42318413 167.39425685 167.36532958 167.33640230
## [16,] -55.60442708 -55.60208825 -55.59974942 -55.59741059 -55.59507176
## [17,]  0.08129767  0.08130229  0.08130691  0.08131153  0.08131614
## [18,]  0.80048382  0.80048482  0.80048582  0.80048681  0.80048781
## [19,] -14.73486542 -14.73476086 -14.73465630 -14.73455174 -14.73444718
## [20,] -108.10291204 -108.07455008 -108.04618812 -108.01782617 -107.98946421
##           [,6]           [,7]           [,8]           [,9]          [,10]
## [1,]  82.99391471  83.00022696  83.00653922  83.01285148  83.01916374
## [2,] -3.21929416 -3.21929818 -3.21930219 -3.21930621 -3.21931022
```

```
## [3,] 12.99814370 12.99806578 12.99798786 12.99790994 12.99783202
## [4,] -13.73523630 -13.73477092 -13.73430555 -13.73384017 -13.73337480
## [5,] -0.06366278 -0.06363842 -0.06361406 -0.06358970 -0.06356534
## [6,] 2.81391840 2.81384741 2.81377642 2.81370544 2.81363445
## [7,] 5.09202281 5.09197363 5.09192444 5.09187526 5.09182607
## [8,] 24.02375399 24.02320306 24.02265213 24.02210121 24.02155028
## [9,] -0.66783554 -0.66783595 -0.66783636 -0.66783677 -0.66783718
## [10,] 1.67429834 1.67429526 1.67429219 1.67428911 1.67428603
## [11,] 1.70441373 1.70439611 1.70437849 1.70436087 1.70434326
## [12,] 0.50892289 0.50895591 0.50898893 0.50902196 0.50905498
## [13,] 1.09835405 1.09834628 1.09833851 1.09833073 1.09832296
## [14,] -0.33623538 -0.33624052 -0.33624565 -0.33625079 -0.33625592
## [15,] 167.30747502 167.27854775 167.24962047 167.22069319 167.19176592
## [16,] -55.59273293 -55.59039410 -55.58805527 -55.58571644 -55.58337761
## [17,] 0.08132076 0.08132538 0.08133000 0.08133461 0.08133923
## [18,] 0.80048881 0.80048981 0.80049081 0.80049181 0.80049280
## [19,] -14.73434262 -14.73423806 -14.73413350 -14.73402895 -14.73392439
## [20,] -107.96110225 -107.93274029 -107.90437834 -107.87601638 -107.84765442
##
## $lambda
## [1] 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0

mse_test <- numeric(length(lambda_values))
for (i in 1:length(lambda_values)) { # nolint
  lambda <- lambda_values[i]
  result_lasso <- lasso_shooting(X_train, y_train, lambda)
  intercept <- result_lasso$intercept
  beta <- result_lasso$coefficients
  y_pred <- intercept + X_test %*% beta
  mse_test[i] <- mean((y_test - y_pred)^2)
}

# find the best lambda
best_lambda <- lambda_values[which.min(mse_test)]
best_coefficients <- beta_matrix[, which.min(mse_test)]
best_lambda

## [1] 1
```

2. Compare your fit against the lasso implementation from glmnet.

```
result_glmnet <- glmnet(X_train, y_train, alpha = 1, lambda = lambda_vector, standardize = TRUE, intercept = 0)
lambda_min_glmnet <- result_glmnet$lambda.min
lambda_1se_glmnet <- result_glmnet$lambda.1se

## NULL
```

3. Fit also a ridge regression and a least squares regression for the data (you can use here glmnet).

```
cv_fit_lasso <- cv.glmnet(X, y, alpha = 1, nfolds = 10)
lambda_min_lasso <- cv_fit_lasso$lambda.min
best_coef_lasso <- coef(cv_fit_lasso, s = lambda_min_lasso)

cv_fit_ridge <- cv.glmnet(X, y, alpha = 0, nfolds = 10)
lambda_min_ridge <- cv_fit_ridge$lambda.min
best_coef_ridge <- coef(cv_fit_ridge, s = lambda_min_ridge)
lambda_min_ridge
```

```
## [1] 25.52821
```

```
lm_fit <- lm(y ~ ., data = data.frame(X, y))
```

4. Compute the lasso, ridge regression and ls regression predictions for the testing data. Which method gives the better predictions? Interpret all three models and argue about their performances.

```
# our lasso
lasso_model <- lasso_shooting(X_train, y_train, best_lambda)
lasso_pred <- lasso_model$intercept + X_test %*% lasso_model$coefficients

# ridge
ridge_model <- glmnet(X_train, y_train, alpha = 0, lambda = best_lambda, standardize = TRUE)
ridge_pred <- predict(ridge_model, s = best_lambda, newx = X_test)

# ls
ls_model <- lm(y_train ~ ., data = as.data.frame(X_train))
ls_pred <- predict(ls_model, newdata = as.data.frame(X_test))

# mse for each model
mse_lasso <- mean((y_test - lasso_pred)^2)
mse_ridge <- mean((y_test - ridge_pred)^2)
mse_ls <- mean((y_test - ls_pred)^2)
mse_df <- data.frame(
  Model = c("LASSO", "Ridge Regression", "Least Squares"),
  MSE = c(mse_lasso, mse_ridge, mse_ls)
)
kable(mse_df, caption = "Model comparison MSE")
```

Table 2: Model comparison MSE

Model	MSE
LASSO	145975.4
Ridge Regression	136861.1
Least Squares	148819.9

Task 3

Regularized regression means adding a penalty term to the loss function in order to prevent overfitting. This penalty is supposed to penalize large coefficients meaning the model is more likely to generalize easier

Shrinkage is a technique that aims to reduce the coefficients of the model to avoid overfitting and make the model simpler. To remove the coefficients we introduce a penalty term that forces some coefficients to be closer to zero.

The main difference between Lasso and Ridge is the penalty term, where lasso uses the absolute value of beta (L1 regularization), whereas ridge uses beta squared in the penalty term (L2 regularization). Lasso can drive some coefficients to exactly zero as opposed to ridge which may reduce some coefficients closer to zero but not exactly zero, therefore lasso is more focused on feature selection (shrinkage). Ridge on the other hand is more focused on using most if not all coefficients but in a generalized manner, making sure the model does not have very large coefficients.